

RIS Fault Prediction Codebase Documentation

generate_data.py

Description:

This file is responsible for generating synthetic channel data for the RIS (Reconfigurable Intelligent Surface) system.

Working Principle:

It defines the system parameters (like frequencies, distances, number of transmitters and RIS elements) and generates near-field channel matrices considering line-of-sight and noise. It creates balanced datasets of normal and faulty RIS conditions (where a faulty element is represented by a 0.0 reflection coefficient instead of the normal phase shift). Finally, it preprocesses the complex numbers into real/imaginary parts and saves the datasets.

models.py

Description:

Contains the neural network architectures used for predicting faulty RIS elements.

Working Principle:

It defines three models: RISFaultMLP (a classical Multi-Layer Perceptron feedforward neural network), RISFaultCNN (a 1D Convolutional Neural Network), and RISFaultQML (a hybrid Quantum Machine Learning model using PennyLane for the quantum circuit and PyTorch for the classical layers).

plot_results.py

Description:

A script to visualize the training results and performance of the models.

Working Principle:

It generates smooth, realistic-looking mock learning curves (loss and accuracy) over 40 epochs for all three models (MLP, CNN, QML) on both fault detection and fault localization tasks. It plots these curves and saves the output as an image (learning_curves.png).

test_fixed_channel.py

Description:

A testing script designed to evaluate the MLP model on a fixed channel realization with varying noise.

Working Principle:

It fixes the channel matrices to avoid variations from spatial changes, then generates samples with and without faults by adding varying noise. It trains a simple MLP model and outputs its accuracy for fault detection.

test_samples.py

RIS Fault Prediction Codebase Documentation

Description:

A script to test a simple MLP model on a larger dataset generated on the fly.

Working Principle:

Uses the data generator to build a dataset of 1000 samples per class. It trains a basic feedforward neural network on this dataset for 50 epochs to detect whether a fault exists, printing the loss periodically and final accuracy.

test_snr.py

Description:

Evaluates the performance of the MLP model under different Signal-to-Noise Ratio (SNR) conditions.

Working Principle:

It loops through a list of SNR values (20, 30, 40 dB), generating a new dataset for each. It then trains and tests an MLP model for each dataset to observe how the detection accuracy is affected by noise levels.

train.py

Description:

The main training pipeline that runs and compares all three models (MLP, CNN, QML) on both tasks.

Working Principle:

It loads the dataset (generating it if missing) and runs two tasks: Binary Fault Detection (perfect vs. faulty) and 16-Class Fault Localization (identifying which element is faulty). It trains all three models on both tasks, calculates evaluation metrics (Accuracy, F1-score), saves a performance comparison text file, and plots learning curves using actual training history.

train_cnn.py

Description:

A standalone training script specifically focused on evaluating the Classical CNN model.

Working Principle:

Similar to train.py, but isolates the CNN training pipeline. It defines a slightly modified CNN architecture, loads data, and runs both fault detection and localization tasks, outputting a performance summary solely for the CNN.

train_mlp.py

Description:

A standalone training script specifically for evaluating the Classical MLP model.

Working Principle:

Isolates the MLP training pipeline. It runs the fault detection and localization tasks using only the MLP

RIS Fault Prediction Codebase Documentation

architecture and outputs a performance summary for the MLP model.

train_qml.py

Description:

A standalone training script for evaluating the Hybrid Quantum Machine Learning (QML) model.

Working Principle:

Isolates the QML training pipeline. It runs the fault detection and localization tasks using the hybrid PennyLane/PyTorch architecture. Since QML is computationally heavier, it uses fewer epochs, and then outputs a performance summary for the QML model.